

Native handles and file streams

Document #: P1759R5
Date: 2023-02-12
Project: Programming Language C++
Audience: Library
Reply-to: Elias Kosunen
[<isocpp@eliaskosunen.com>](mailto:isocpp@eliaskosunen.com)

Contents

- [1 Abstract](#)
- [2 Revision History](#)
 - [2.1 R5](#)
 - [2.2 R4](#)
 - [2.3 R3](#)
 - [2.4 R2](#)
 - [2.5 R1](#)
 - [2.6 R0](#)
- [3 Design Discussion](#)
 - [3.1 Implementation-definedness of native handles and presence of them](#)
- [4 Impact On the Standard and Existing Code](#)
- [5 Implementation](#)
- [6 Technical Specifications](#)
 - [6.1 Wording notes](#)
 - [6.2 Feature test macro](#)
 - [6.3 Wording](#)
- [7 Acknowledgements](#)
- [8 References](#)

§ 1 Abstract

This paper proposes adding a new typedef to standard file streams: `native_handle_type`. This type is an alias to whatever type the platform uses for its file descriptors: `int` on POSIX, `HANDLE (void*)` on Windows, and something else on other platforms. This type is a non-owning handle and has generally sane semantics: default constructability, trivial copyability and it's standard layout.

Alongside this, this paper proposes adding a concrete member function: `.native_handle()`, returning a `native_handle_type`, to the following class templates:

- `basic_filebuf`
- `basic_ifstream`
- `basic_ofstream`
- `basic_fstream`

§ 2 Revision History

§ 2.1 R5

- Remove Motivation, Scope and Prior Art
- Update design discussion
 - Add `std::stacktrace_handle` as a type that has a `.native_handle()`, and comparisons to it
- Update wording:
 - Update wording to reference [N4928]
 - *Expects -> Preconditions*
 - Other minor changes based on LEWG feedback

§ 2.2 R4

- Update wording:
 - `native_handle_type constexpr default constructor -> default constructor`
 - Add wording as to how the handle returned by `.native_handle()` is valid only when the file is open
- Update wording to reference the latest standard draft [N4892]

§ 2.3 R3

- Add `std::condition_variable` and [P2146R2] to list of standard types having a `.native_handle()` member function
- Update wording to reference the latest standard draft [N4849], and update references to other P-numbered papers

- Change paper title

§ 2.4 R2

- Minor touches to wording
 - Refine requirements on `native_handle_type` (remove `equality_comparable`, add `constexpr` default constructability)
 - Fix some broken references using section numbers in the WD
 - Update reference to the WD
- Editorial fixes

§ 2.5 R1

- Make `native_handle_type` be standard layout
- Add precondition (`is_open() == true`) to `.native_handle()`
- Add feature test macro `__cpp_lib_fstream_native_handle`
- Fix errors with opening the file with POSIX APIs in Motivation (see, we need this paper, fstreams are easier to open correctly!)
- Add additional motivating use case in vectored/scatter-gather IO
- Regular -> regular

Incorporate LEWGI feedback from Cologne (July 2019):

- Move to a member function and member typedef
- Make `native_handle` return value not be mandated to be unique
- Add note about how the presence of the members is required, and not implementation-defined (like for `thread`)

§ 2.6 R0

Initial revision.

§ 3 Design Discussion

§ 3.1 Implementation-definedness of native handles and presence of them

The wording related to native handles in 32.2.3 [[thread.req.native](#)] is as follows:

Several classes described in this Clause have members `native_handle_type` and `native_handle`. The presence of these members and their semantics is implementation-defined. [*Note:* These members allow implementations to

provide access to implementation details. Their names are specified to facilitate portable compile-time detection. Actual use of these members is inherently non-portable. — *end note*]

In more plain terms, the presence of native handles in `std::thread`, `std::mutex`, and `std::condition_variable`, is implementation-defined, without a way to query, whether they are provided (except with SFINAE magic). This design is deemed bad by the author and by LEWG, so this paper aims to not do that again.

The wording related to native handles in [\[stacktrace.entry.obs\]](#) is as follows:

The semantics of this function are implementation-defined.

Remarks: Successive invocations of the `native_handle` function for an unchanged `stacktrace_entry` object return identical values.

While this design is a huge step up compared to threads, the author would like to have more strict normative guarantees for this facility.

This paper proposes to:

- set requirements for `native_handle_type`: semiregularity, trivial copyability, standard-layout
- define (semi-normatively), what a *native handle* for a file means, and how it behaves

§ 4 Impact On the Standard and Existing Code

This proposal is a pure library extension, requiring no changes to the core language. It would cause no existing conforming code to break.

§ 5 Implementation

Implementing this paper should be a relatively trivial task.

Although all implementations surveyed (libstdc++, libc++ and MSVC) use `FILE*` instead of native file descriptors in their `basic_filebuf` implementations, these platforms provide facilities to get a native handle from a `FILE*`; `fileno` on POSIX, and `_fileno` + `_get_osfhandle` on Windows. The following reference implementations use these.

For libstdc++ on Linux:

```
template <class CharT, class Traits>
class basic_filebuf : public basic_streambuf<CharT, Traits> {
    // ...
    using native_handle_type = int;
    // ...
}
```

```

    native_handle_type native_handle() {
        assert(is_open());
        // _M_file (__basic_file<char>) has a member function for this
        // purpose
        return _M_file.fd();
        // ::fileno(_M_file.file()) could also be used
    }
    // ...
}

```

For libc++ on Linux:

```

template <class CharT, class Traits>
class basic_filebuf : public basic_streambuf<CharT, Traits> {
    // ...
    using native_handle_type = int;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // __file_ is a FILE*
        return ::fileno(__file_)
    }
    // ...
}

```

For MSVC:

```

template <class CharT, class Traits>
class basic_filebuf : public basic_streambuf<CharT, Traits> {
    // ...
    using native_handle_type = HANDLE;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // _Myfile is a FILE*
        auto cfile = ::_fileno(_Myfile);
        // _get_osfhandle returns intptr_t, which can be cast to HANDLE
        (void*)
        return static_cast<HANDLE>(::_get_osfhandle(cfile));
    }
    // ...
}

```

For all of these cases, implementing `.native_handle()` for `ifstream`, `ofstream` and `fstream` is trivial:

```

template <class CharT, class Traits>
class basic_ifstream : public basic_istream<CharT, Traits> {
    // ...
    using native_handle_type =
        typename basic_filebuf<CharT, Traits>::native_handle_type;
    // ...
    native_handle_type native_handle() {
        return rdbuf()->native_handle();
    }
}

```

```
};  
  
// Repeat for ofstream and fstream
```

§ 6 Technical Specifications

§ 6.1 Wording notes

The wording is based on [N4928].

§ 6.2 Feature test macro

This paper proposes adding a feature test macro, called `__cpp_lib_fstream_native_handle`.

§ 6.3 Wording

§ 6.3.1 Add the following section into *File-based streams* [file.streams]

This section is to come between 29.9.1 [fstream.syn] and 29.9.2 [filebuf].

Note to editor: Replace the ? with the appropriate section number. As of [N4928], that would be 31.10.2.

Also, replace the ? in the Note with the appropriate note number.

???. Native handles [file.native]

Several classes described in this section have a member `native_handle_type`.

The type `native_handle_type` represents a platform-specific *native handle* to a file. It is trivially copyable and standard layout, and models semiregular.

[*Note* ?: For operating systems based on POSIX, `native_handle_type` is `int`. For Windows-based operating systems, `native_handle_type` is `HANDLE`.
— *end note*]

§ 6.3.2 Modify Class template *basic_filebuf* [filebuf]

```
namespace std {  
    template<class charT, class traits = char_traits<charT>>  
    class basic_filebuf : public basic_streambuf<charT, traits> {  
    public:  
        using char_type    = charT;  
        using int_type      = typename traits::int_type;  
        using pos_type      = typename traits::pos_type;  
        using off_type      = typename traits::off_type;  
        using traits_type   = traits;
```

```

+     using native_handle_type = implementation-defined; // see
        [file.native]

    // ...

    // [filebuf.members], members
    bool is_open() const;
    basic_filebuf* open(const char* s, ios_base::openmode mode);
    basic_filebuf* open(const filesystem::path::value_type* s,
        ios_base::openmode mode); // wide systems only;
        see 31.10.1
    basic_filebuf* open(const string& s,
        ios_base::openmode mode);
    basic_filebuf* open(const filesystem::path& s,
        ios_base::openmode mode);
    basic_filebuf* close();
+     native_handle_type native_handle();

    // ...
}
}

```

§ 6.3.3 Modify Class *template basic_filebuf* [filebuf]

Note to editor: Replace the ? in the note number below with the appropriate note number.

An instance of `basic_filebuf` behaves as described in [filebuf] provided `traits::pos_type` is `fpos<traits::state_type>`. Otherwise the behavior is undefined.

The underlying file of a `basic_filebuf` has an associated value of type `native_handle_type`, called the native handle of that file. This native handle can be obtained with the member function `native_handle`. Whether a value of `native_handle_type` not obtained by calling `native_handle()` may be a valid native handle is implementation-defined. Whether the associated native handle is unique for each file, is implementation-defined.

For any opened `basic_filebuf` `f`, the native handle returned by `f.native_handle()` is invalidated when `f.close()` is called, or `f` is destroyed.

[*Note* ?: The type `native_handle_type`, and the member function `native_handle`, of file-based streams are always defined. This is consistent with native handles of `stacktrace_entry` [stacktrace.entry], but differs from `thread`, `mutex` and `condition_variable` [thread.req.native], the presence of which is implementation-defined. — *end note*]

In order to support file I/O and multibyte/wide character conversion, conversions are performed using members of a facet, referred to as `a_codecvt` in the following subclauses, obtained as if by...

§ 6.3.4 Add to the end of *Member functions* [filebuf.members]

This would come after the definition of `basic_filebuf::close()`, which occupies paragraphs 8-10.

```
native_handle_type native_handle();
```

Preconditions: `is_open()` is true.

Throws: Nothing.

Returns: The native handle associated with this file.

§ 6.3.5 Modify Class template *basic_ifstream* [ifstream]

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ifstream : public basic_istream<charT, traits> {
    public:
        using char_type      = charT;
        using int_type        = typename traits::int_type;
        using pos_type        = typename traits::pos_type;
        using off_type        = typename traits::off_type;
        using traits_type     = traits;
+       using native_handle_type =
+           typename basic_filebuf<charT, traits>::native_handle_type;

        // ...

        // [ifstream.members], members
        basic_filebuf<charT, traits>* rdbuf() const;
+       native_handle_type native_handle();

        bool is_open() const;
        // ...
    }
}
```

§ 6.3.6 Add to *Member functions* [ifstream.members] after p1

This would come between the definitions of `basic_ifstream::rdbuf()` (p1) and `basic_ifstream::is_open()` (p2, now p3).

```
native_handle_type native_handle();
```

Effects: Equivalent to: `return rdbuf()->native_handle();`.

§ 6.3.7 Modify Class template *basic_ofstream* [ofstream]

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ofstream : public basic_ostream<charT, traits> {
    public:
```



```

    using char_type    = charT;
    using int_type     = typename traits::int_type;
    using pos_type     = typename traits::pos_type;
    using off_type     = typename traits::off_type;
    using traits_type  = traits;
+   using native_handle_type =
+       typename basic_filebuf<charT, traits>::native_handle_type;

    // ...

    // [ofstream.members], members
    basic_filebuf<charT, traits>* rdbuf() const;
+   native_handle_type native_handle();

    bool is_open() const;
    // ...
}

```

§ 6.3.8 Add to *Member functions* [ofstream.members] after p1

This would come between the definitions of `basic_ofstream::rdbuf()` (p1) and `basic_ofstream::is_open()` (p2, now p3).

```
native_handle_type native_handle();
```

Effects: Equivalent to: `return rdbuf()->native_handle();`.

§ 6.3.9 Modify *Class template* `basic_fstream` [fstream]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_fstream : public basic_istream<charT, traits> {
    public:
        using char_type    = charT;
        using int_type     = typename traits::int_type;
        using pos_type     = typename traits::pos_type;
        using off_type     = typename traits::off_type;
        using traits_type  = traits;
+       using native_handle_type =
+           typename basic_filebuf<charT, traits>::native_handle_type;

        // ...

        // [fstream.members], members
        basic_filebuf<charT, traits>* rdbuf() const;
+       native_handle_type native_handle();
+
        bool is_open() const;
        // ...
    }
}

```

§ 6.3.10 Add to *Member functions* [fstream.members] after p1

This would come between the definitions of `basic_fstream::rdbuf()` (p1) and `basic_fstream::is_open()` (p2, now p3).

```
native_handle_type native_handle();
```

Effects: Equivalent to: `return rdbuf()->native_handle();`.

§ 7 Acknowledgements

Thanks to Jonathan Wakely for reviewing the wording for R3 of this paper.

Thanks to Niall Douglas for feedback, encouragement and ambitious suggestions for this paper.

Thanks to the rest of the co-authors of [P1750R1] for the idea after cutting this functionality out, especially to Jeff Garland for providing a heads-up about a possible ABI-break that I totally would've missed, even though it ended up being a non-issue.

Thanks to Michael Park for his paper markup framework [mpark/wg21].

§ 8 References

[mpark/wg21] mpark/wg21 on GitHub.

<https://github.com/mpark/wg21>

[N4849] Richard Smith. 2020-01-14. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4849>

[N4892] Richard Smith. 2022. Working Draft, Standard for Programming Language C++.

<https://wg21.link/N4892>

[N4928] Thomas Köppe. 2022. Working Draft, Standard for Programming Language C++.

<https://wg21.link/N4928>

[P1750R1] Klemens Morgenstern, Jeff Garland, Elias Kosunen, and Fatih Bakir. 2019. A Proposal to Add Process Management to the C++ Standard Library.

<https://wg21.link/p1750r1>

[P2146R2] Amanda Kornoushenko. 2020. Modern `std::byte` stream IO for C++.

<https://wg21.link/p2146r2>